

STAT 8020 R Session 12: Time Series Analysis I

Whitney Huang, Clemson University

Contents

Time Series Data	1
Lake Huron Time Series	1
Atmospheric CO ₂ Concentrations	2
STL Decomposition of the CO ₂ Series	3
U.S. Monthly Unemployment Rates	4
Time Series Models	4
ARMA: Autocovariance Function (ACF)	5
ARMA Models: Partial Autocorrelation Function (PACF)	6
Lake Huron Case Study: Modeling Annual Water Levels	8
Step 1: Explore the Time Series	8
Step 2: Examine the ACF and PACF	9
Step 3: Estimate and Remove the Trend	10
Step 4: Re-examine the Detrended Series	11
Step 5: Fit Candidate Models	11
AR(2)	13
ARMA(2, 1)	15
Step 6: Compare Models Using AIC	17
AR(2) Fitting and Forecasting	18

Time Series Data

This section introduces several time series datasets. Through these examples, we will explore key characteristics of time series data, including trends, seasonality, cycles, and long-term changes over time.

Lake Huron Time Series

The `LakeHuron` dataset contains annual measurements of the average water level of Lake Huron (in feet) from 1875 to 1972.

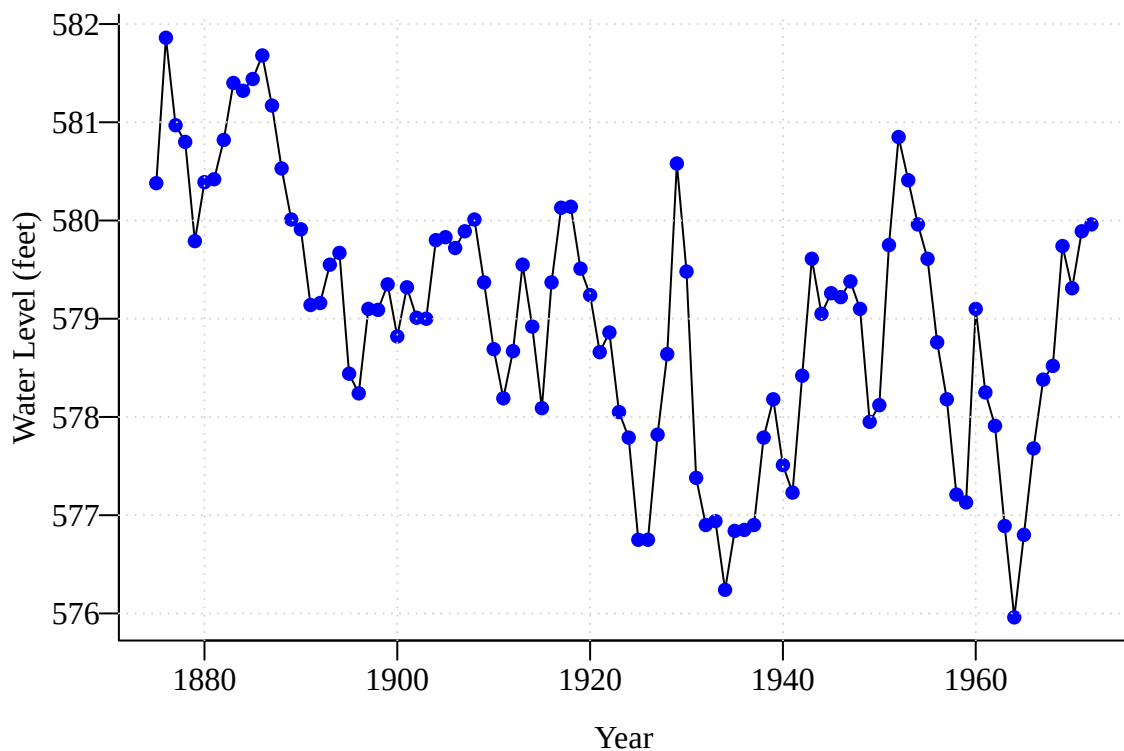
```
# Adjust plotting parameters for a cleaner display
par(mar = c(3.2, 3.2, 0.5, 0.5), mgp = c(2, 0.5, 0),
    bty = "L", las = 1, family = "serif")

# Load the Lake Huron water level data
data(LakeHuron)

# Plot the time series
plot(LakeHuron, ylab = "Water Level (feet)", xlab = "Year")

# Add observations as points to emphasize individual measurements
points(LakeHuron, pch = 16, col = "blue")

# Add a background grid for easier interpretation
grid()
```



Atmospheric CO₂ Concentrations

The `co2` dataset contains monthly atmospheric carbon dioxide concentrations (ppm) measured at Mauna Loa Observatory in Hawaii. This is one of the most famous environmental time series and exhibits both a long-term increasing trend and a strong annual seasonal cycle.

```
# Load the monthly CO2 concentration data
data(co2)

# Set graphical parameters
par(mar = c(3.8, 4, 0.8, 0.6), las = 1, mgp = c(2, 1, 0), family = "serif")
```

```

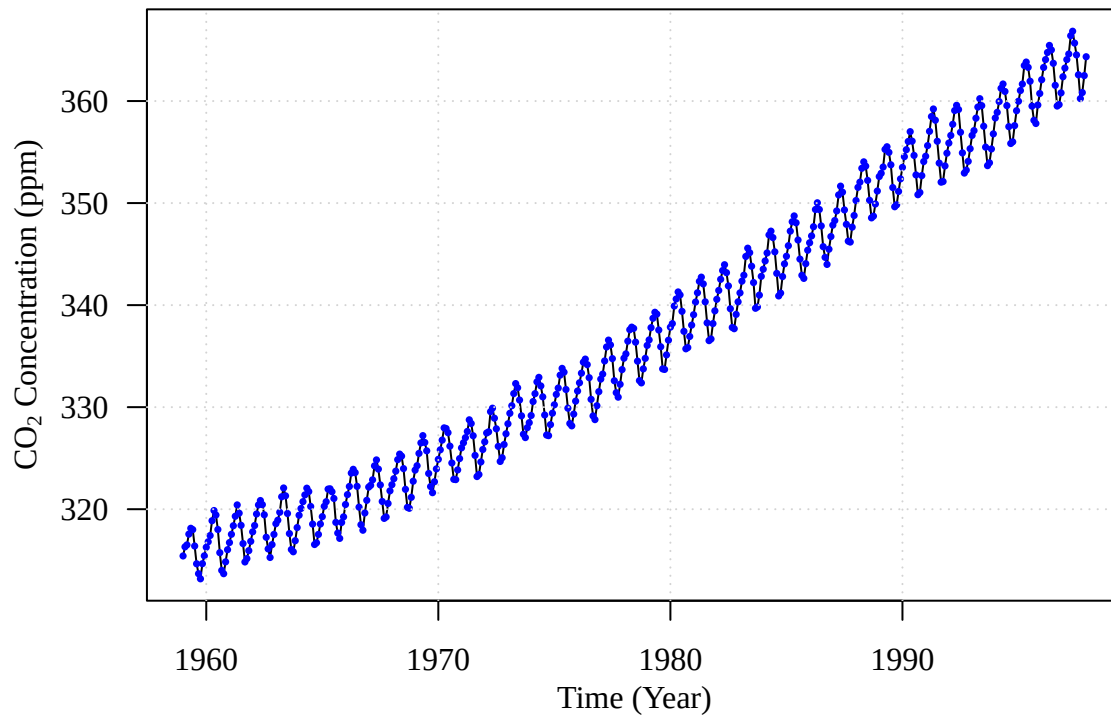
# Plot the time series
plot(co2, xlab = "", ylab = "")

# Overlay observed values
points(co2, pch = 16, col = "blue", cex = 0.5)

# Add customized axis labels
mtext("Time (Year)", side = 1, line = 2)
mtext(expression(paste("CO"[2], " Concentration (ppm)")), side = 2, line = 2.5, las = 0)

# Add a background grid
grid()

```



STL Decomposition of the CO₂ Series

To better understand the underlying structure of the CO₂ series, we perform a Seasonal-Trend decomposition using Loess (STL). STL separates the observed series into three components:

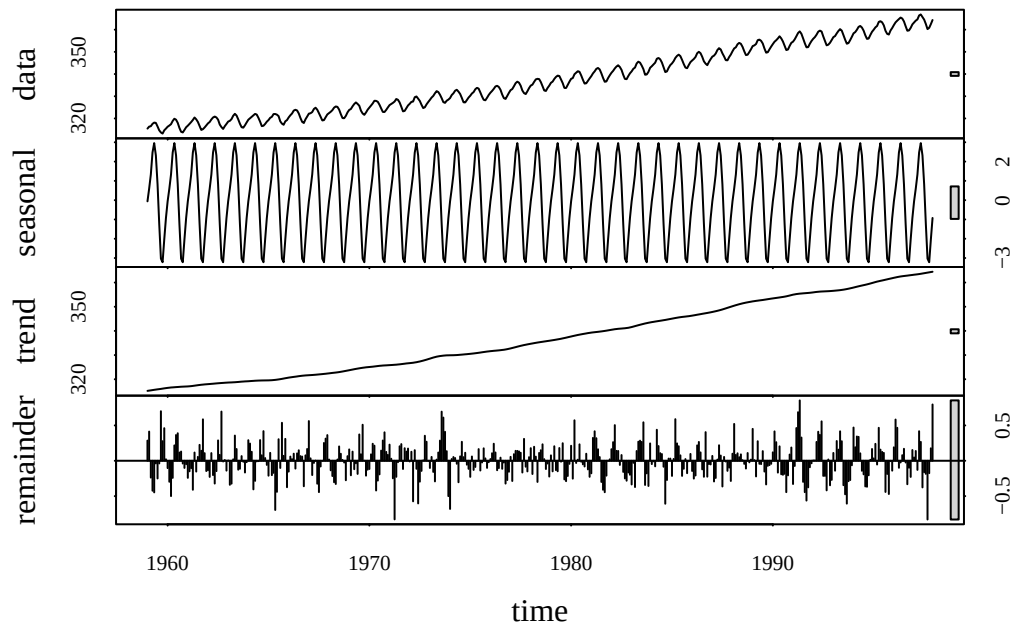
- **Trend:** Long-term changes over time
- **Seasonal:** Recurring within-year patterns
- **Remainder:** Short-term fluctuations not explained by the trend or seasonal components

```

# Perform Seasonal-Trend decomposition using Loess (STL)
stl_fit <- stl(co2, s.window = "periodic")

# Plot the decomposed components
par(mar = c(4, 3.6, 0.8, 0.6), mgp = c(2, 1, 0), family = "serif")
plot(stl_fit)

```



U.S. Monthly Unemployment Rates

The following example uses monthly U.S. unemployment rates obtained directly from the Federal Reserve Economic Data (FRED) database. Economic time series often exhibit business cycles, structural changes, and periods of increased volatility.

```
# Load the quantmod package for accessing financial and economic data
library(quantmod)

# Download U.S. unemployment rate data from FRED
getSymbols("UNRATE", src = "FRED")

# Display the first and last few observations
head(UNRATE)
tail(UNRATE)

# Plot the unemployment rate series
chartSeries(UNRATE, name = "U.S. Monthly Unemployment Rate")
```

Time Series Models

In this section, we use simulated time series to compare the behavior of several basic stationary time series models:

- White noise
- Autoregressive model: AR(1)
- Moving average model: MA(1)
- Autoregressive moving average model: ARMA(1,1)

The goal is to understand how the autocorrelation function (ACF) and partial autocorrelation function (PACF) help us identify appropriate time series models.

ARMA: Autocovariance Function (ACF)

The autocorrelation function measures the correlation between observations separated by different time lags. It is especially useful for identifying dependence patterns in a time series.

For model identification:

- White noise should have almost no significant autocorrelation.
- An AR(1) model usually has an ACF that decays gradually.
- An MA(1) model usually has an ACF that cuts off after lag 1.
- An ARMA(1,1) model usually has an ACF that decays gradually but may have a more complex pattern.

```
# Set a random seed so that the simulation results are reproducible
set.seed(123)

# Define the length of each simulated time series
n <- 200

# Simulate white noise: independent normal random variables
WN <- rnorm(n)

# Simulate an AR(1) model with phi = 0.8
# The current value depends on the previous value plus random noise
AR <- arima.sim(n = n, model = list(ar = 0.8))

# Simulate an MA(1) model with theta = 0.5
# The current value depends on the current and previous random shocks
MA <- arima.sim(n = n, model = list(ma = 0.5))

# Simulate an ARMA(1,1) model with phi = 0.8 and theta = 0.5
# This model combines autoregressive and moving average dependence
ARMA <- arima.sim(n = n, model = list(ar = 0.8, ma = 0.5))
# Arrange the plots in a 4-by-2 layout:
# each row shows one time series and its corresponding ACF
par(mfrow = c(4, 2), mar = c(3.6, 3.6, 1.2, 0.6), las = 1, mgp = c(2, 1, 0),
    family = "serif")

# White noise series and ACF
plot(1:n, WN, type = "l", xlab = "", ylab = "")
mtext("White Noise")

acf(WN, xlab = "", ylab = "", main = "")
mtext("ACF")

# AR(1) series and ACF
plot(1:n, AR, type = "l", xlab = "", ylab = "")
mtext(expression(AR(1) ~ ":" ~ phi == 0.8))
```

```

acf(AR, xlab = "", ylab = "", main = "")

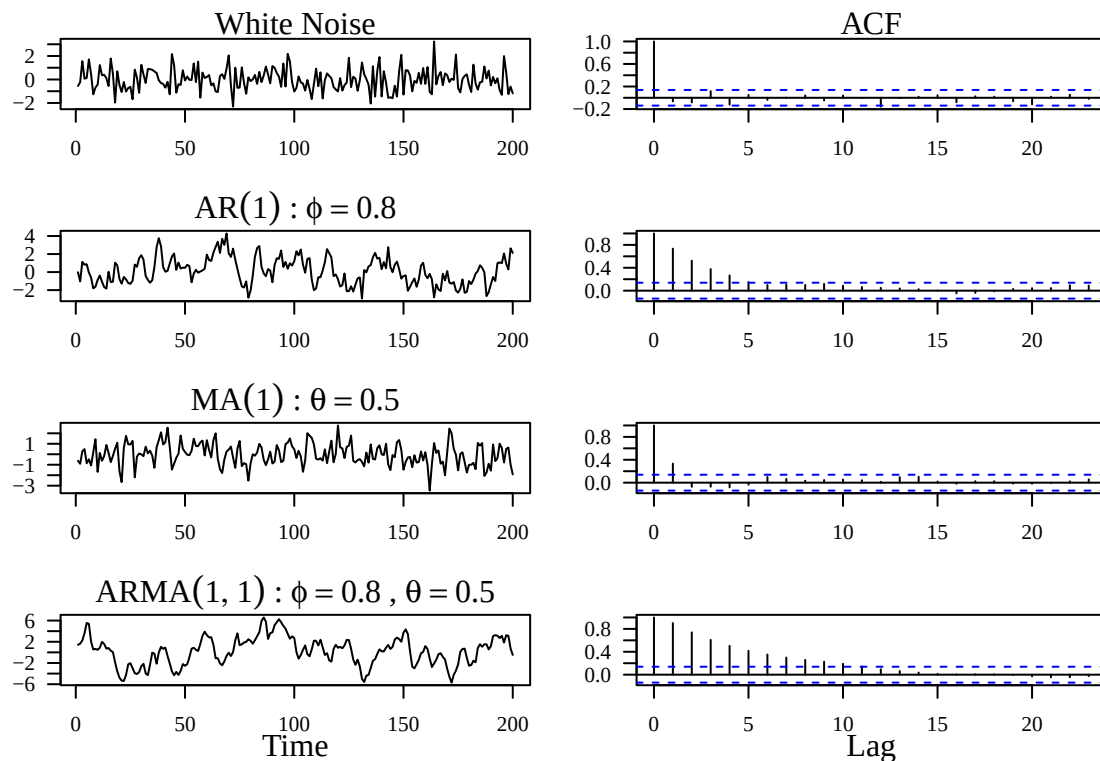
# MA(1) series and ACF
plot(1:n, MA, type = "l", xlab = "", ylab = "")
mtext(expression(MA(1) ~ ":" ~ theta == 0.5))

acf(MA, xlab = "", ylab = "", main = "")

# ARMA(1,1) series and ACF
plot(1:n, ARMA, type = "l", xlab = "", ylab = "")
mtext(expression(ARMA(1,1) ~ ":" ~ phi == 0.8 ~ "," ~ theta == 0.5))
mtext("Time", side = 1, line = 2)

acf(ARMA, xlab = "", ylab = "", main = "")
mtext("Lag", side = 1, line = 2)

```



ARMA Models: Partial Autocorrelation Function (PACF)

The partial autocorrelation function measures the correlation between a time series and its lagged values after removing the linear effects of the intermediate lags.

For model identification:

- White noise should have no clear PACF pattern.
- An AR(1) model usually has a PACF that cuts off after lag 1.
- An MA(1) model usually has a PACF that decays gradually.

- An ARMA(1,1) model often has both ACF and PACF gradually decaying.

```
# Arrange plots in a 4 × 2 layout:
# each row contains a time series and its corresponding PACF
par(mfrow = c(4, 2), mar = c(3.6, 3.6, 1.2, 0.6), las = 1,
    mgp = c(2, 1, 0), family = "serif")

# White noise series and PACF
plot(1:n, WN, type = "l", xlab = "", ylab = "")
mtext("White Noise")

pacf(WN, xlab = "", ylab = "", main = "")
mtext("PACF")

# AR(1) series and PACF
plot(1:n, AR, type = "l", xlab = "", ylab = "")
mtext(expression(AR(1) ~ ":" ~ phi == 0.8))

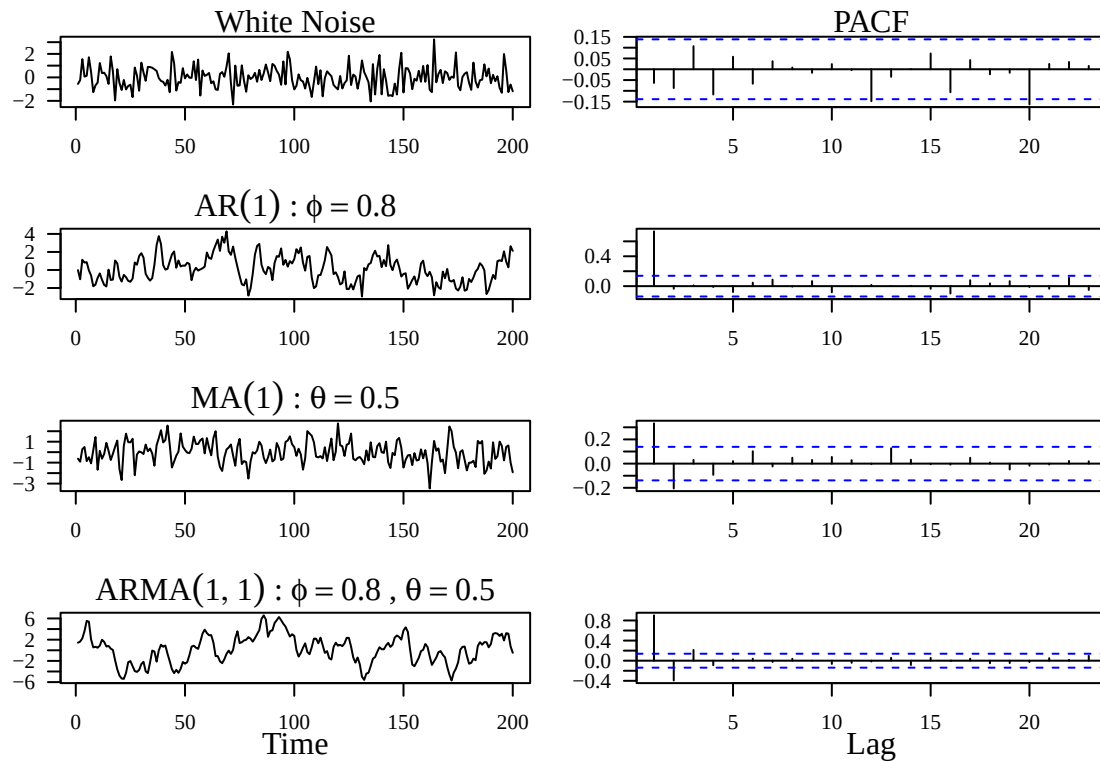
pacf(AR, xlab = "", ylab = "", main = "")

# MA(1) series and PACF
plot(1:n, MA, type = "l", xlab = "", ylab = "")
mtext(expression(MA(1) ~ ":" ~ theta == 0.5))

pacf(MA, xlab = "", ylab = "", main = "")

# ARMA(1,1) series and PACF
plot(1:n, ARMA, type = "l", xlab = "", ylab = "")
mtext(expression(ARMA(1,1) ~ ":" ~ phi == 0.8 ~ "," ~ theta == 0.5))
mtext("Time", side = 1, line = 2)

pacf(ARMA, xlab = "", ylab = "", main = "")
mtext("Lag", side = 1, line = 2)
```



Lake Huron Case Study: Modeling Annual Water Levels

In this case study, we will apply the complete ARMA modeling workflow to the Lake Huron water level data.

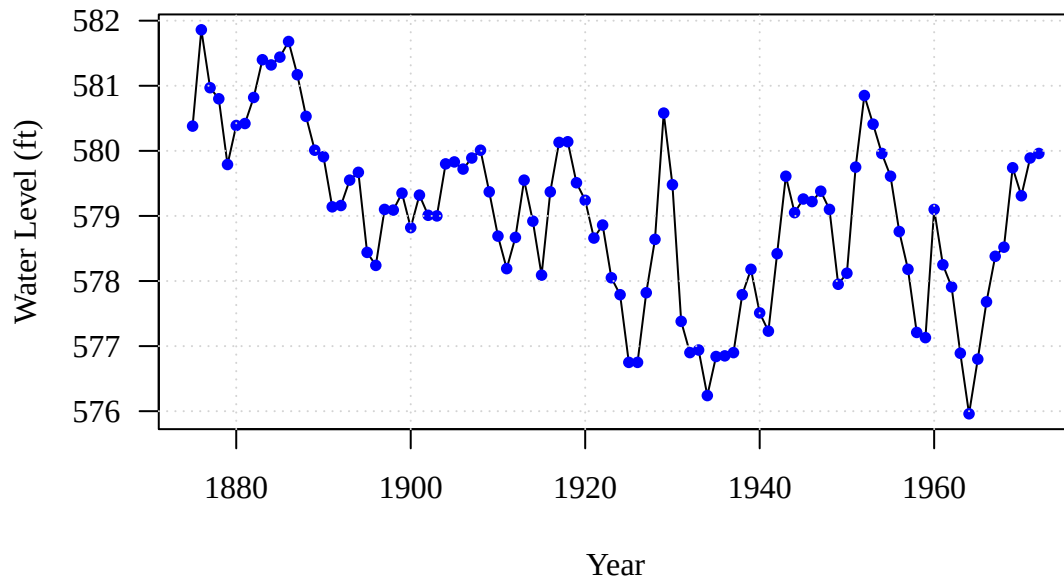
By the end of this case study, you should be able to:

1. Visualize the data.
2. Estimate/Remove the trend.
3. Examine the ACF and PACF.
4. Fit candidate ARMA models.
5. Compare models and generate forecasts.

Step 1: Explore the Time Series

```
# Observation years
years <- time(LakeHuron)

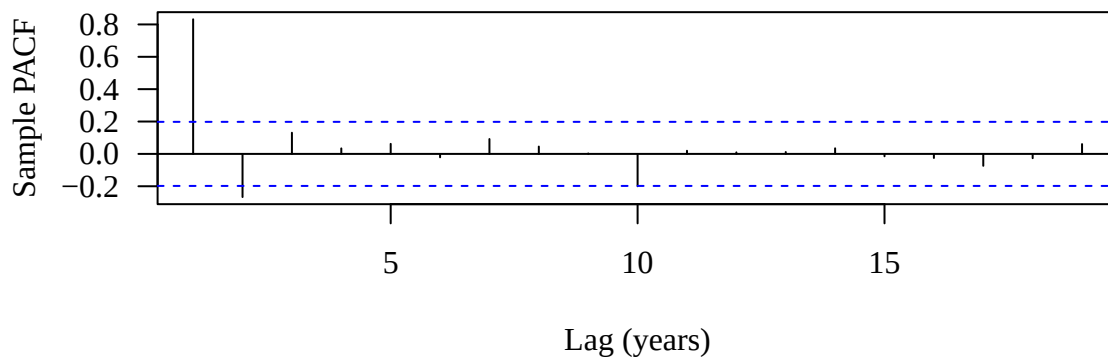
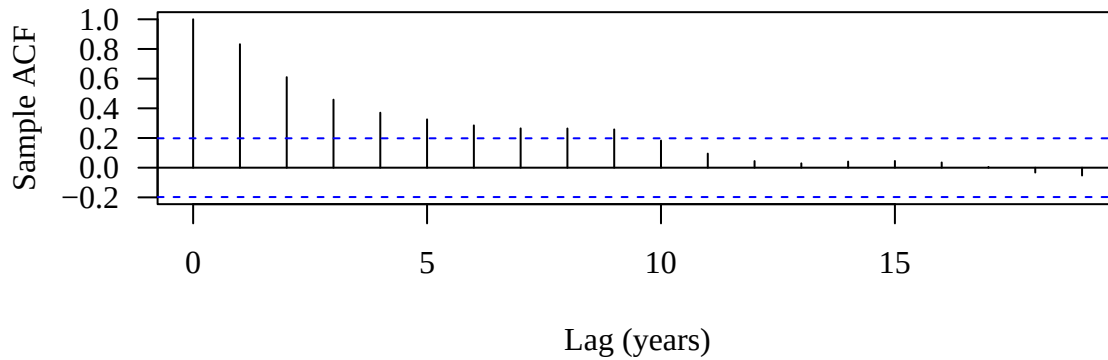
# Time series plot
plot(LakeHuron, ylab = "Water Level (ft)", xlab = "Year", las = 1, family = 'serif')
points(LakeHuron, pch = 16, col = "blue", cex = 0.8)
grid()
```



The series exhibits a gradual downward trend, suggesting that detrending may be necessary before fitting a stationary ARMA model.

Step 2: Examine the ACF and PACF

```
par(mfrow = c(2, 1), mar = c(4, 4, 1, 1), las = 1, family = 'serif')
acf(LakeHuron, xlab = "Lag (years)", ylab = "Sample ACF", main = "")
pacf(LakeHuron, xlab = "Lag (years)", ylab = "Sample PACF", main = "")
```



The slowly decaying ACF suggests the presence of a trend and serial dependence.

Step 3: Estimate and Remove the Trend

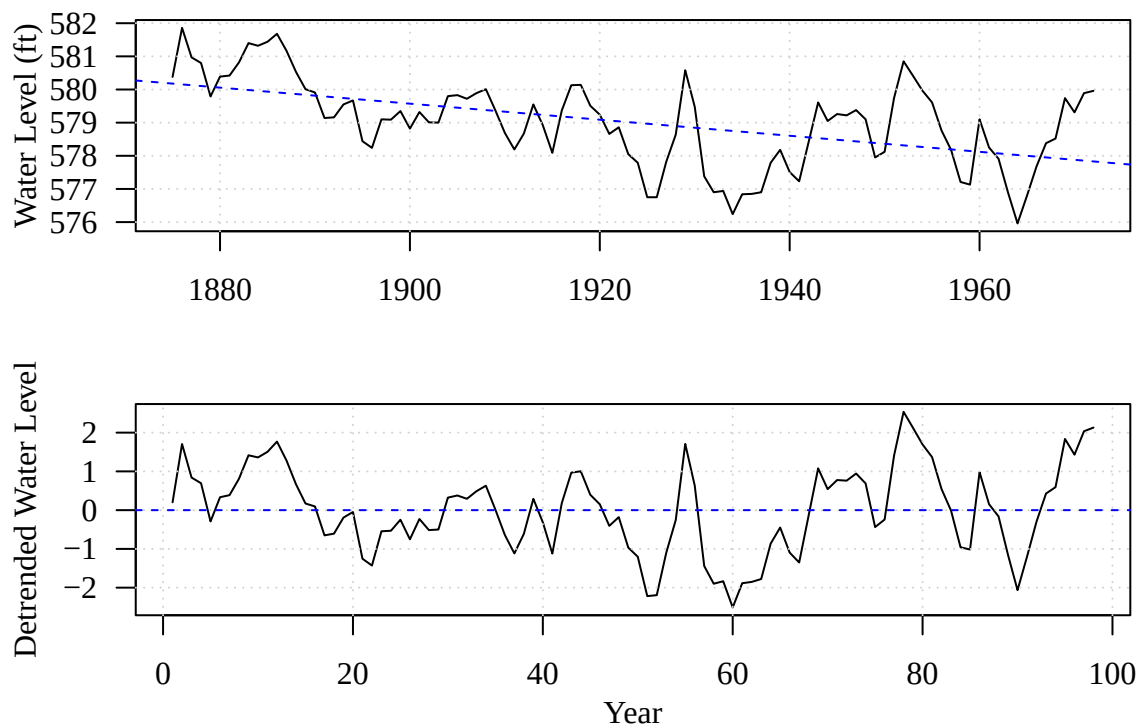
```
# Fit a linear trend
trend.fit <- lm(LakeHuron ~ years)

# Detrended series
deTrend <- resid(trend.fit)

par(mfrow = c(2, 1), mar = c(3.5, 3.5, 1, 0.6), las = 1, family = "serif")

# Original series and fitted trend
plot(LakeHuron, ylab = "", xlab = "")
grid()
abline(trend.fit, col = "blue", lty = 2)
mtext("Water Level (ft)", side = 2, line = 2.4, las = 0)

# Detrended series
plot(1:98, deTrend, type = "l", ylab = "", xlab = "")
grid()
abline(h = 0, col = "blue", lty = 2)
mtext("Year", side = 1, line = 2)
mtext("Detrended Water Level", side = 2, line = 2.4, las = 0)
```



The residuals from the linear regression represent the detrended series.

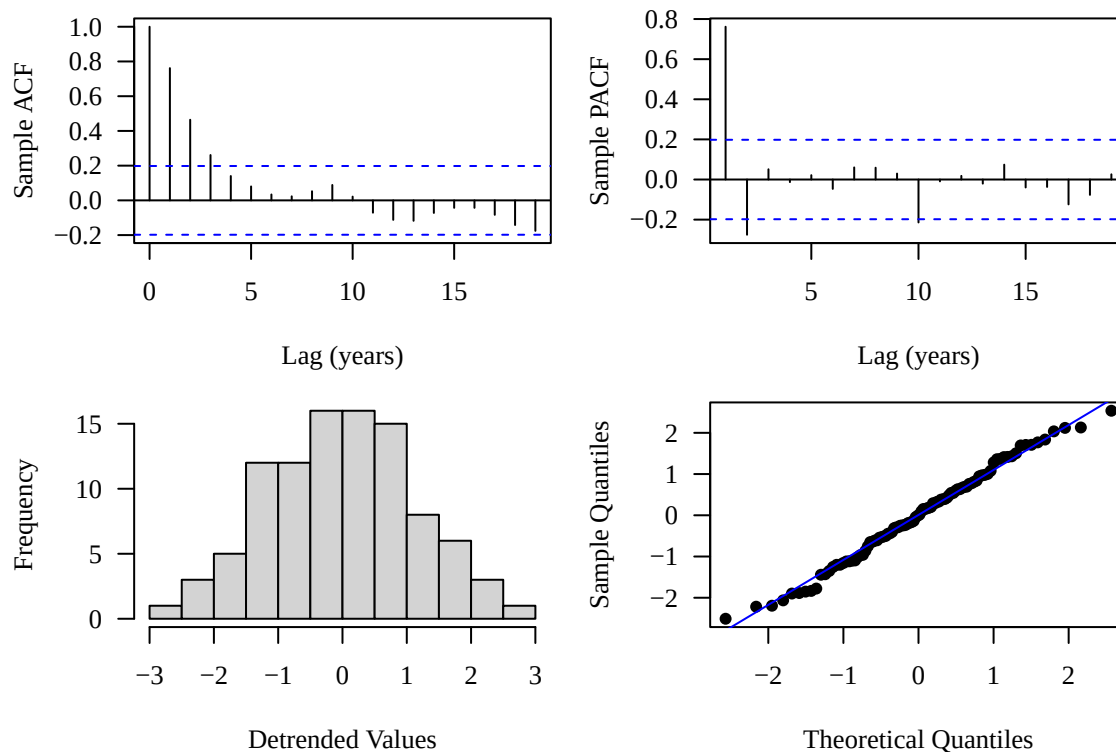
Step 4: Re-examine the Detrended Series

```
par(mfrow = c(2, 2), mar = c(4, 4, 1, 1), las = 1, family = 'serif')

acf(deTrend, xlab = "Lag (years)", ylab = "Sample ACF", main = "")
pacf(deTrend, xlab = "Lag (years)", ylab = "Sample PACF", main = "")

hist(deTrend, breaks = 15, main = "", xlab = "Detrended Values")

qqnorm(deTrend, main = "", pch = 16)
qqline(deTrend, col = "blue")
```



The ACF and PACF are used to identify candidate ARMA models, while the histogram and Q-Q plot provide a quick assessment of normality.

Step 5: Fit Candidate Models

Instead of immediately relying on automated procedures, we fit several plausible models and compare them. Candidate models:

1. AR(1)
2. AR(2)
3. ARMA(2,1)

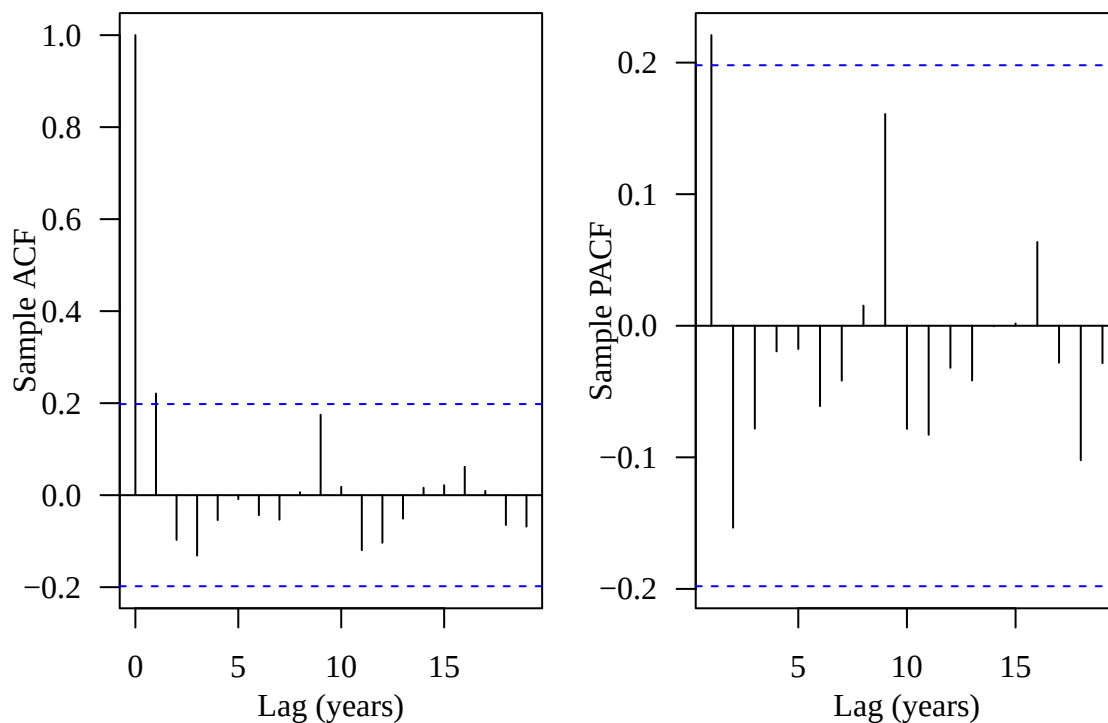
```
(ar1.model <- arima(deTrend, order = c(1, 0, 0), method = "ML"))
```

AR(1)

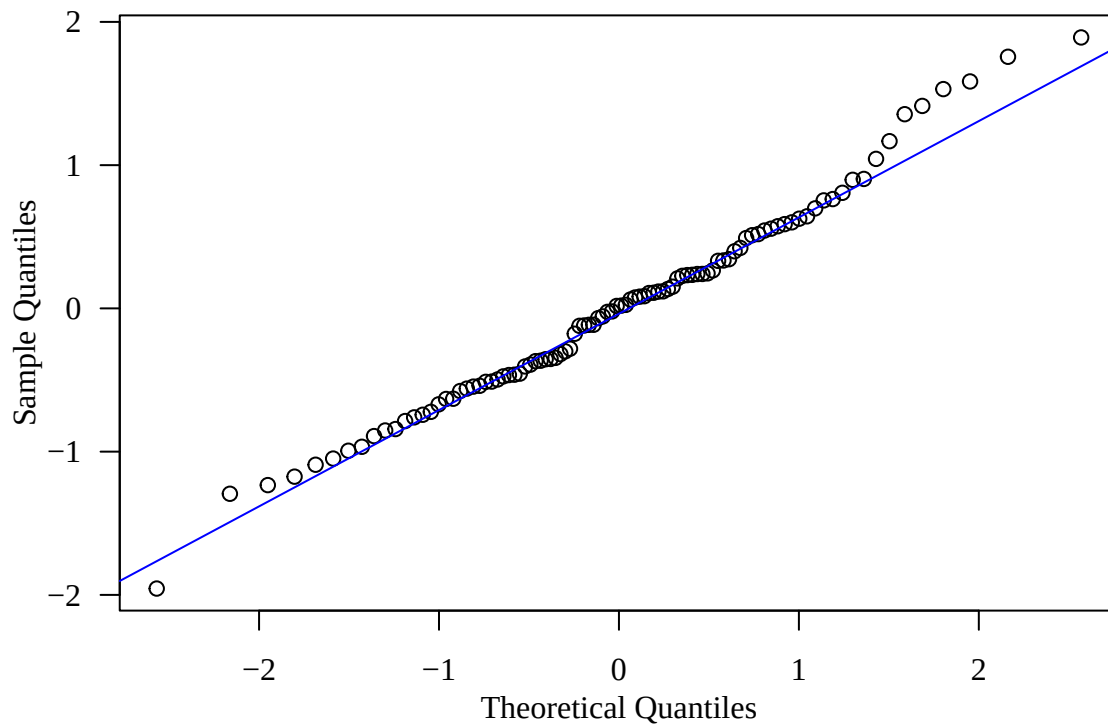
```
##
## Call:
## arima(x = deTrend, order = c(1, 0, 0), method = "ML")
##
## Coefficients:
##          ar1  intercept
##      0.7829    0.0799
## s.e. 0.0634    0.3179
##
## sigma^2 estimated as 0.4972:  log likelihood = -105.29,  aic = 216.58
```

```
ar1.resids <- resid(ar1.model)
```

```
# Residual diagnostics
par(mfrow = c(1, 2), mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 1, 0), family = "serif", las = 1)
acf(ar1.resids, ylab = "Sample ACF", xlab = "Lag (years)")
pacf(ar1.resids, ylab = "Sample PACF", xlab = "Lag (years)")
```



```
par(mfrow = c(1, 1), mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 1, 0), family = "serif", las = 1)
qqnorm(ar1.resids, main = "")
qqline(ar1.resids, col = "blue")
```



```
# Ljung-Box test
Box.test(ar1.resids, lag = 5, fitdf = 1, type = "Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: ar1.resids
## X-squared = 7.9867, df = 4, p-value = 0.09207
```

```
(ar2.model <- arima(deTrend, order = c(2, 0, 0), method = "ML"))
```

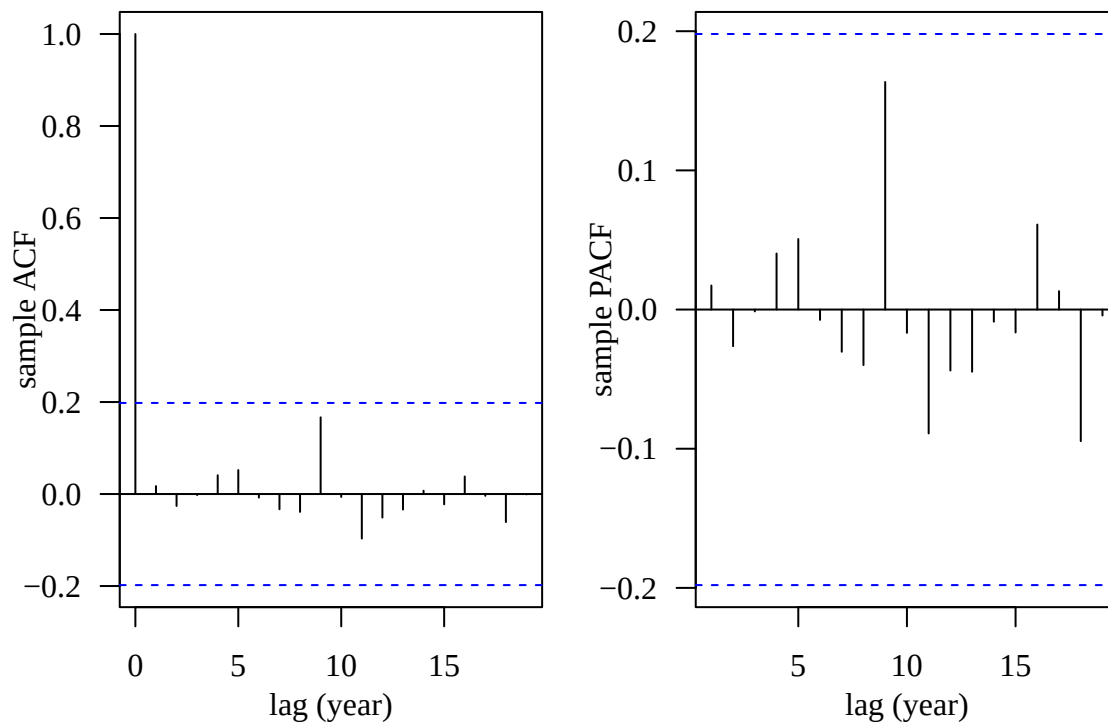
AR(2)

```
##
## Call:
## arima(x = deTrend, order = c(2, 0, 0), method = "ML")
##
## Coefficients:
##          ar1      ar2  intercept
##       1.0047  -0.2919    0.0197
## s.e.  0.0977   0.1004    0.2350
##
## sigma^2 estimated as 0.4571:  log likelihood = -101.25,  aic = 210.5
```

```
## calculate the residuals
ar2.resids <- resid(ar2.model)
```

```
## Sample ACF and PACF of the residuals
```

```
par(mfrow = c(1, 2), mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 1, 0), family = "serif", las = 1)
acf(ar2.resids, ylab = "sample ACF", xlab = "lag (year)")
pacf(ar2.resids, ylab = "sample PACF", xlab = "lag (year)")
```



```
## Test for time dependence for the residuals
```

```
Box.test(ar2.resids, lag = 5, fitdf = 2, type = "Ljung-Box")
```

```
##
```

```
## Box-Ljung test
```

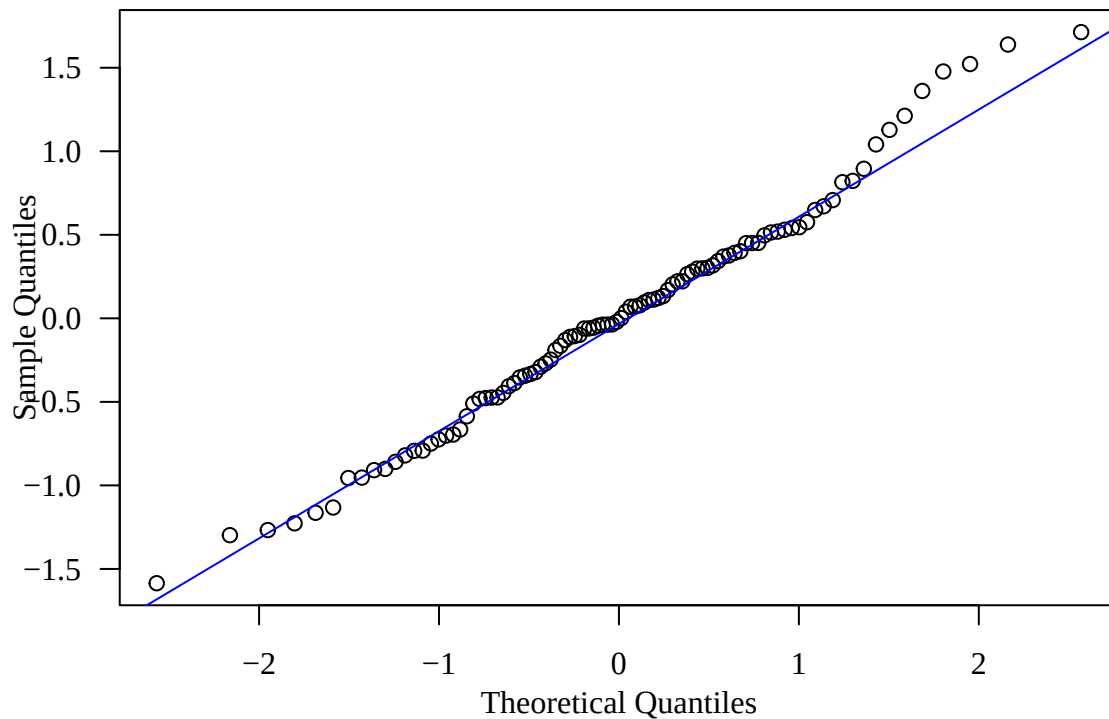
```
##
```

```
## data: ar2.resids
```

```
## X-squared = 0.55962, df = 3, p-value = 0.9056
```

```
## Normal Q-Q plot for the residuals
```

```
par(mfrow = c(1, 1), mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 1, 0), family = "serif", las = 1)
qqnorm(ar2.resids, main = ""); qqline(ar2.resids, col = "blue")
```

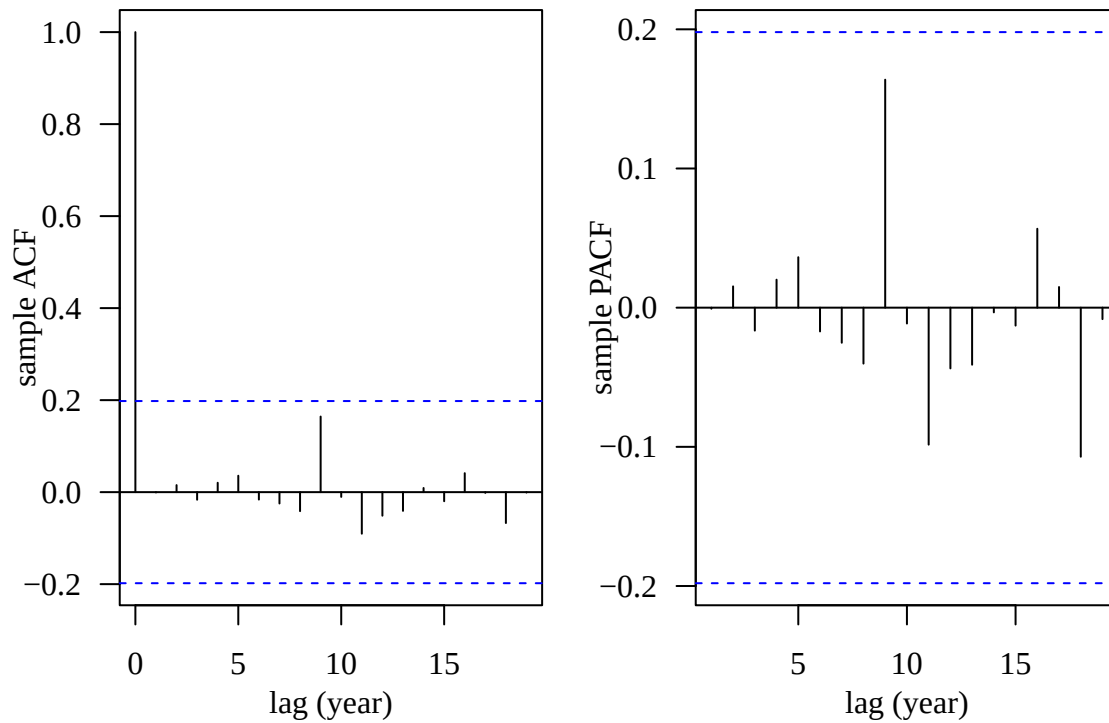


```
(arma21.model <- arima(deTrend, order = c(2, 0, 1)))
```

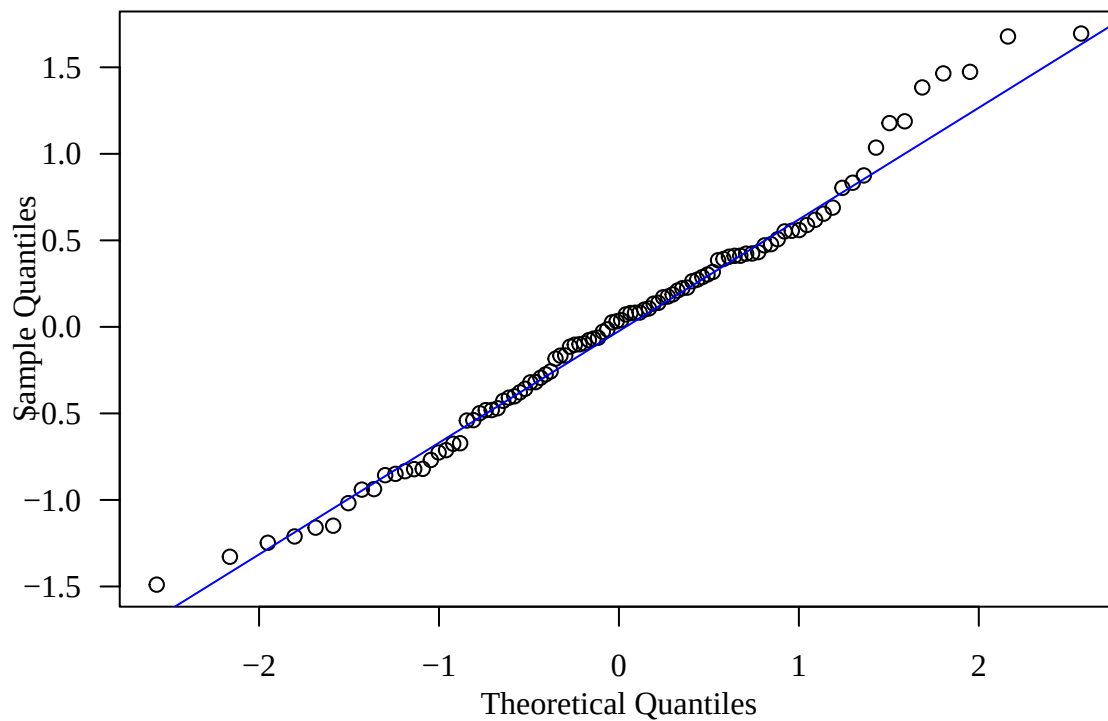
ARMA(2, 1)

```
##
## Call:
## arima(x = deTrend, order = c(2, 0, 1))
##
## Coefficients:
##      ar1      ar2      ma1  intercept
##      0.8374 -0.1622  0.1846    0.0245
## s.e.  0.3180  0.2621  0.3180    0.2452
##
## sigma^2 estimated as 0.4556:  log likelihood = -101.09,  aic = 212.18

## calculate the residuals
arma21.resids <- resid(arma21.model)
## Sample ACF and PACF of the residuals
par(mfrow = c(1, 2), mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 1, 0), family = "serif", las = 1)
acf(arma21.resids, ylab = "sample ACF", xlab = "lag (year)")
pacf(arma21.resids, ylab = "sample PACF", xlab = "lag (year)")
```



```
## Normal Q-Q plot for the residuals
par(mfrow = c(1, 1), mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 1, 0), family = "serif", las = 1)
qqnorm(arma21.resids, main = ""); qqline(arma21.resids, col = "blue")
```



```
## Test
Box.test(arma21.resids, lag = 5, fitdf = 3, type = "Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: arma21.resids
## X-squared = 0.2297, df = 2, p-value = 0.8915
```

```
AIC(ar1.model); AIC(ar2.model); AIC(arma21.model)
```

Step 6: Compare Models Using AIC

```
## [1] 216.5835
```

```
## [1] 210.5032
```

```
## [1] 212.1784
```

```
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo
```

```
auto.arima(deTrend, trace = T)
```

```
##
## ARIMA(2,0,2) with non-zero mean : 215.0455
## ARIMA(0,0,0) with non-zero mean : 304.222
## ARIMA(1,0,0) with non-zero mean : 216.8388
## ARIMA(0,0,1) with non-zero mean : 235.4585
## ARIMA(0,0,0) with zero mean      : 302.1373
## ARIMA(1,0,2) with non-zero mean : 212.7747
## ARIMA(0,0,2) with non-zero mean : 218.2478
## ARIMA(1,0,1) with non-zero mean : 210.9477
## ARIMA(2,0,1) with non-zero mean : 212.8306
## ARIMA(2,0,0) with non-zero mean : 210.9333
## ARIMA(3,0,0) with non-zero mean : 212.7787
## ARIMA(3,0,1) with non-zero mean : Inf
## ARIMA(2,0,0) with zero mean      : 208.7655
## ARIMA(1,0,0) with zero mean      : 214.7735
## ARIMA(3,0,0) with zero mean      : 210.569
## ARIMA(2,0,1) with zero mean      : 210.6186
## ARIMA(1,0,1) with zero mean      : 208.7891
## ARIMA(3,0,1) with zero mean      : Inf
##
## Best model: ARIMA(2,0,0) with zero mean

## Series: deTrend
## ARIMA(2,0,0) with zero mean
```

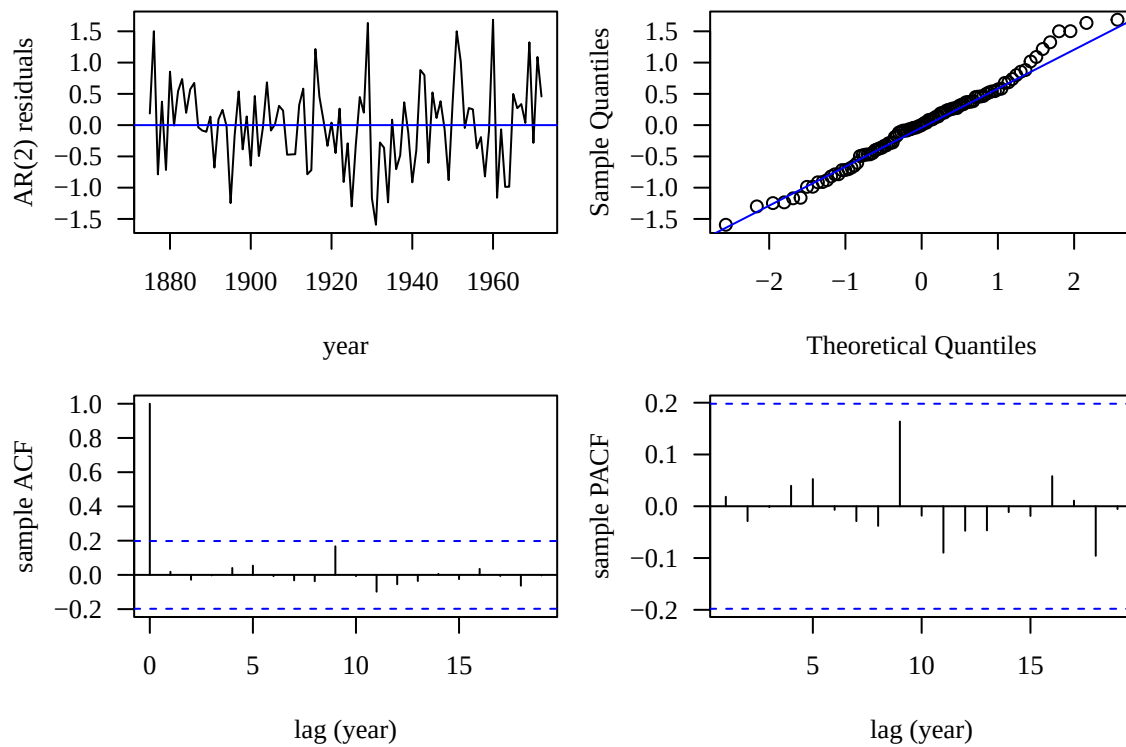
```
##
## Coefficients:
##          ar1      ar2
##      1.0050 -0.2925
## s.e.  0.0976  0.1002
##
## sigma^2 = 0.4667: log likelihood = -101.26
## AIC=208.51  AICc=208.77  BIC=216.27
```

AR(2) Fitting and Forecasting

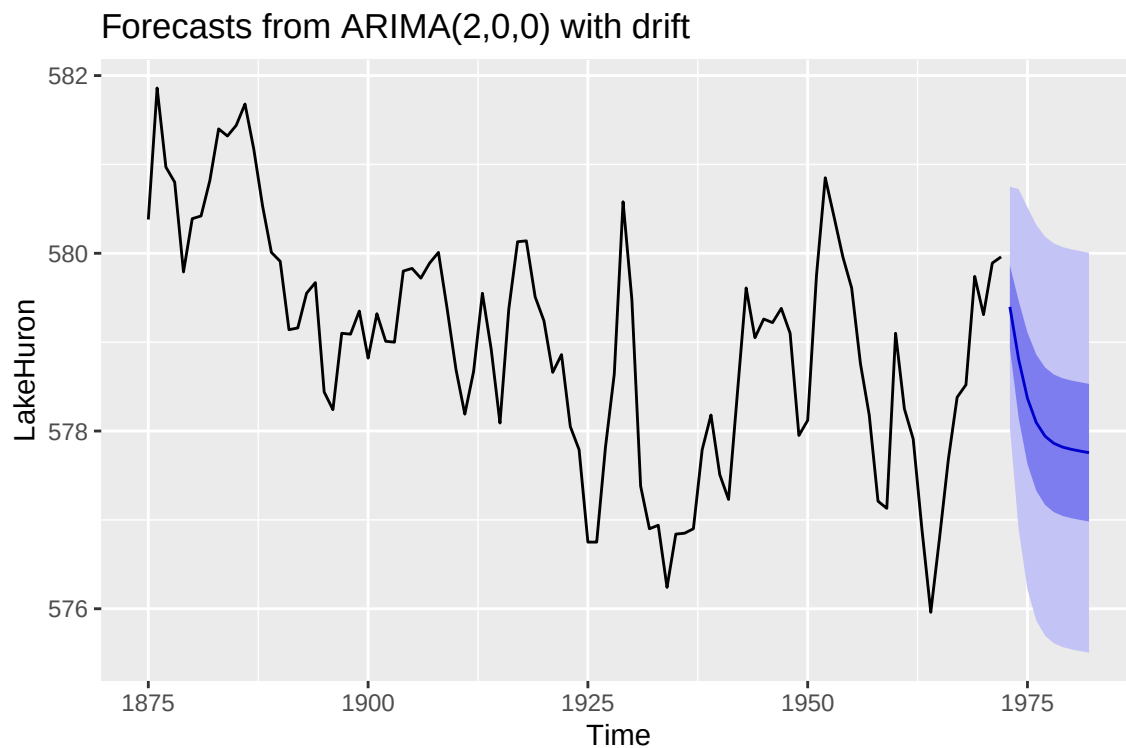
```
(fit <- Arima(LakeHuron, order = c(2, 0, 0), include.drift = T))
```

```
## Series: LakeHuron
## ARIMA(2,0,0) with drift
##
## Coefficients:
##          ar1      ar2 intercept    drift
##      1.0048 -0.2913  580.0915 -0.0216
## s.e.  0.0976  0.1004    0.4636  0.0081
##
## sigma^2 = 0.476: log likelihood = -101.2
## AIC=212.4  AICc=213.05  BIC=225.32
```

```
par(mfrow = c(2, 2), mar = c(4.1, 4, 1, 0.8), las = 1, family = "serif")
res <- fit$residuals
plot(res, type = "l", xlab = "year", ylab = "AR(2) residuals")
abline(h = 0, col = "blue")
qqnorm(res, main = ""); qqline(res, col = "blue")
acf(res, ylab = "sample ACF", xlab = "lag (year)")
pacf(res, ylab = "sample PACF", xlab = "lag (year)")
```



```
# 10-year-ahead forecasts
autoplot(forecast(fit, h = 10, level = c(50, 95)))
```



The forecast plot shows:

- Point forecasts (expected future lake levels)

- 50% prediction intervals
- 95% prediction intervals

Notice that prediction intervals widen as the forecast horizon increases, reflecting growing uncertainty about future observations.